AMRITA VISHWA VIDYAPEETHAM

25AIC114

Elements of Computing – 2 (Assembly Programming)

PROJECT TITLE:

“Program Execution & Resource Monitoring System”

GROUP 9:

Dhiren Vasudeva(CB.AI.U4CPS25014)

Pardhav Reddy(CB.AI.U4CPS25059)

Poovya.S(CB.AI.U4CPS25061)

Ch.Venkata Abhiram(CB.AI.U4CPS25012)

Edwin Santosh(CB.AI.U4CPS25045)

Contents of presentation

- Introduction and Motivation
- Problem Statement
- Project Objectives
- Tools & Technologies Used
- Modular System Architecture
- Engine: Trace Hook Execution
- Modular: Resource Telemetry
- Perforamnce Visualization
- Intelligent Insights
- Comparative Results
- Individual Contributions
- Limitations & Future Work
- Conclusion

AMRITA VISHWA VIDYAPEETHAM

Introduction

Understanding how programs execute internally is difficult for many learners.

Although concepts such as instruction execution, control flow, and memory usage are taught theoretically, the actual runtime behavior of programs is not easily visible.

This project aims to create a system that can trace program execution and monitor system resources during runtime.

Problem Statement

S.No	Paper	Main Idea	Limitation
1	CrossCode: Multi-level Visualization of Program Execution (2023)	Visualizes program execution at different abstraction levels to help understand program behavior.	Focuses mainly on visualization, not system resource monitoring.
2	Educational Program Visualization Using Execution Information (2024)	Uses execution traces to visualize program operations like array reads and writes.	Designed mainly for educational purposes.
3	Graph-Based Program Visualization (Cal Poly Thesis)	Uses graphs to represent program flow and function interactions.	Does not monitor runtime performance metrics.
4	Software Visualization (Stony Brook Report)	Explains techniques to visualize software structure and execution behavior.	Mostly theoretical with limited implementation.
5	Program Visualization in Computer Science Education	Shows that visualization improves understanding of programming concepts.	Focused on learning rather than performance monitoring.

Objectives

The main objectives of this project are:

- Monitor program execution step-by-step
- Track function calls and returns
- Observe variable value changes during runtime
- Measure CPU usage and RAM usage
- Measure execution time and memory consumption
- Visualize execution behavior using graphs

Tools & Technologies Used

1. Programming Language

- Python

2. Libraries Used

- sys: Used for execution tracing using sys.settrace().
- os: Used for file handling and locating the user program.
- tracemalloc: Tracks memory allocation and peak memory usage.
- psutil: Used for monitoring CPU usage, RAM usage, and process memory.
- time: Used to measure execution time.
- matplotlib: Used to generate performance graphs for analysis.

Improvements after Mid-Review

- Interactive UI (Streamlit-based IDE)
- Real-time execution tracking
- Graph visualization
- Call stack visualization
- Variable timeline tracking
- Program comparison system
- Insights engine

UI Transformation

Before:

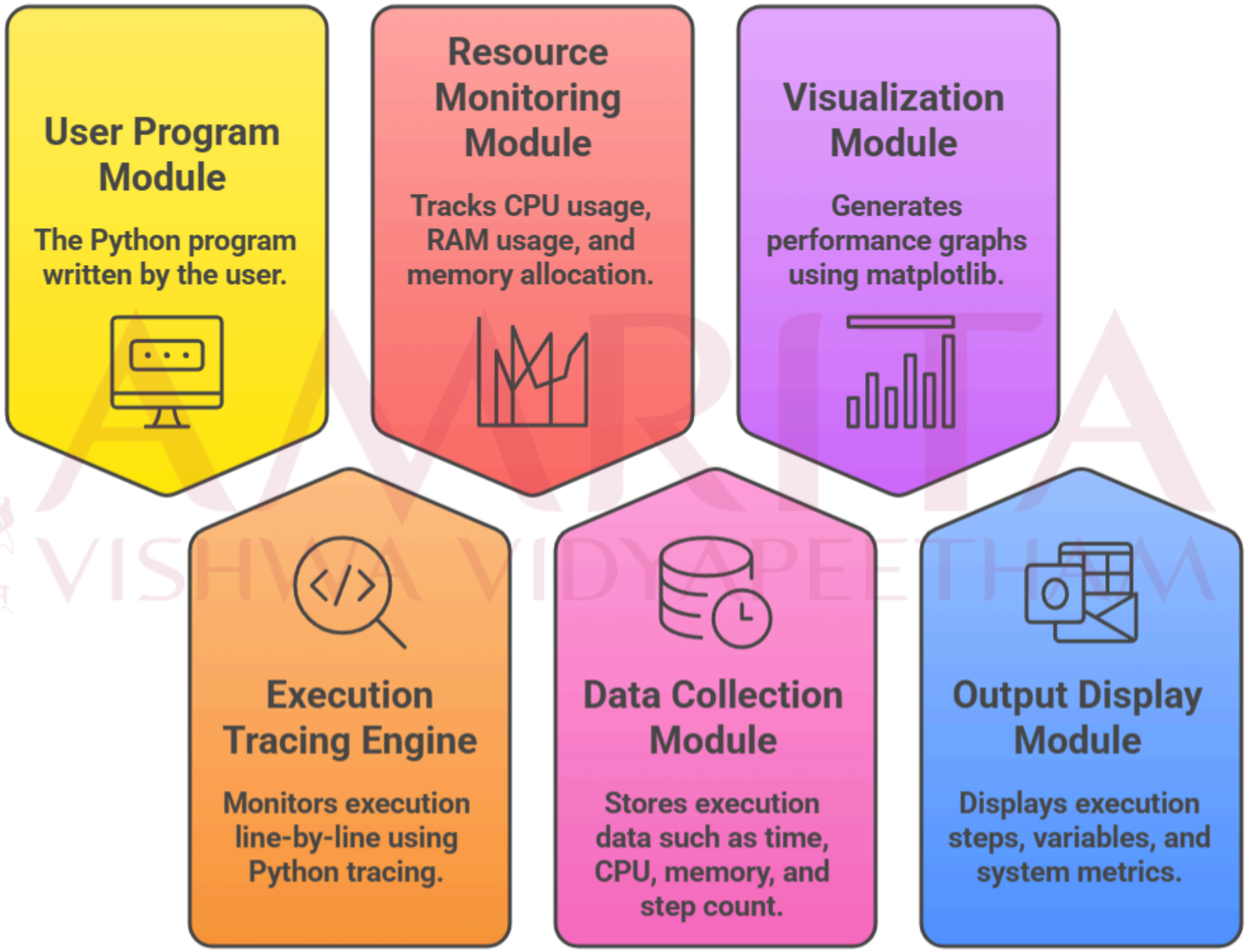
- Terminal-based output
- Hard to understand

After:

- Full IDE interface
- Visual panels:
- Code editor
- Console
- Graphs
- Variables
- Call stack

AMRITA VISHWA VIDYAPEETHAM

System Architecture



AMRITA VISHWA VIDYAPEETHAM
MATA AMRITANANDAMAYI MATH
श्रद्धावान् लभते ज्ञानम्

Resource Monitoring Module

CPU Telemetry

Captures process CPU percentage to identify computationally expensive loops.

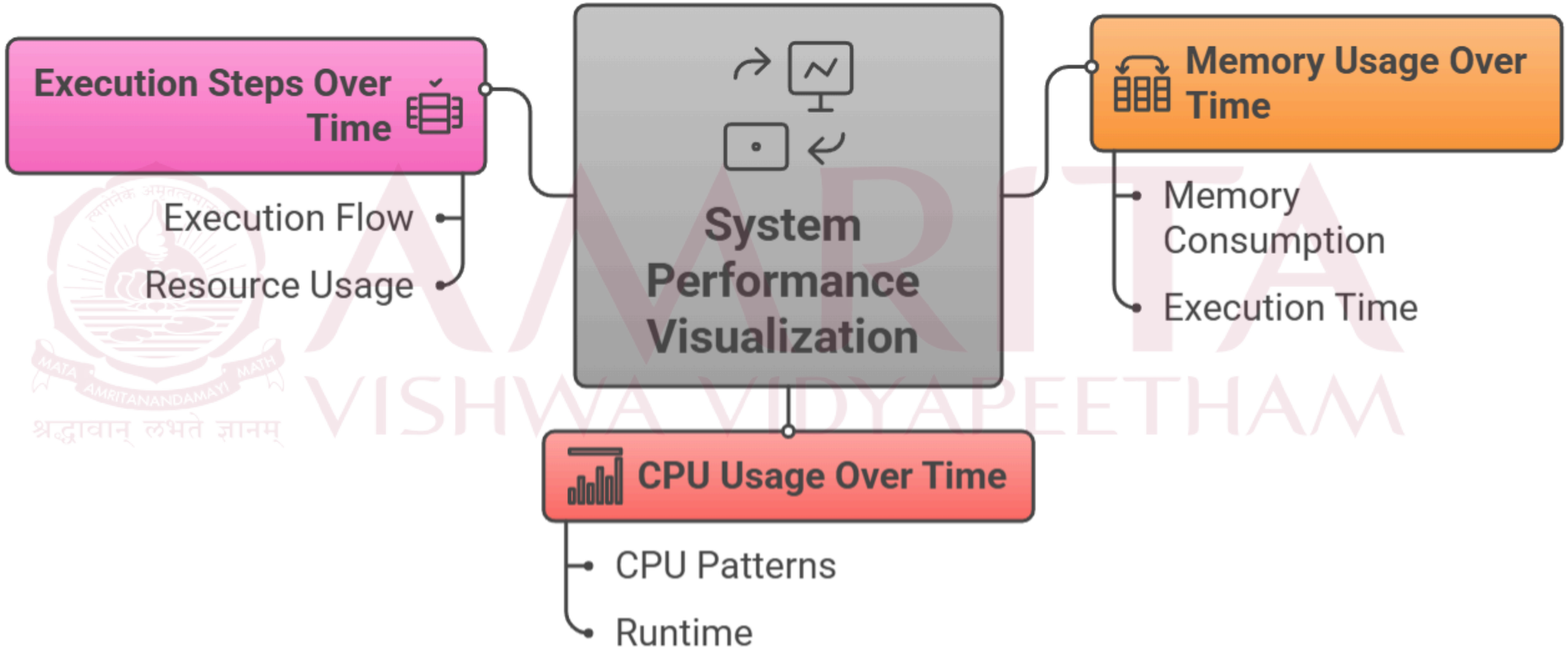
Memory Snapshot

Uses tracemalloc to monitor peak heap size during data structure growth.

Latency Metrics

Calculates delta time between steps to find hidden delays in code execution.

Performance Visualization



AMRITA VISHWA VIDYAPEETHAM

System Visualization

AMRITA
VISHWA VIDYAPEETHAM

Multi-language Integration

- We extended the system to support:
- Execution of programs in multiple languages

Current focus:

- Python (fully supported)
- C/C++ (planned/integration phase)
- Java (future scope)

AMRITA VISHWA VIDYAPEETHAM

Multi-language Design Approach

- Use subprocess-based execution
- Language-specific runners
- Unified output format
- Same visualization pipeline

AMRITA VISHWA VIDYAPEETHAM

Individual Contributions

Member	Contribution
Poovya S	Research and literature review on program execution analysis and monitoring techniques
Dhiren	System architecture design and execution tracing implementation
Pardhav Reddy	Runtime behavior analysis, execution flow monitoring, and variable tracking
Ch.Venkata Abhiram	Resource monitoring module development (CPU, RAM, memory tracking) and code integration
Edwin Santosh	Performance graph visualization, system testing, and documentation

AMRITA VISHWA VIDYAPEETHAM

Conclusion

The proposed system successfully demonstrates how programs execute internally and how system resources are utilized during execution.

By combining execution tracing with resource monitoring and visualization, the system helps users understand program behavior and performance more effectively.

AMRITA VISHWA VIDYAPEETHAM
श्रद्धावान् लभते ज्ञानम्

THANK YOU

AMRITA
VISHWA VIDYAPEETHAM